

Ejercicios para casa

Cómo trabajar este documento

20 ejercicios cortos, organizados por epígrafe, para practicar por tu cuenta las estructuras de datos. Sigue usando tu carpeta de ejercicios con su propio entorno virtual — un archivo `.py` por ejercicio.

- Todas las variables, declaradas, inicializadas y anotadas con su tipo (`list`, `tuple`, `dict`, según corresponda).
- Puedes seguir usando todo lo de los Bloques 1 y 2: condicionales, bucles, funciones... espera, funciones todavía no (llegan en el Bloque 4). Por ahora, todo en el cuerpo principal del script.
- Comprueba tú mismo el resultado ejecutando el programa y revisando que VS Code no marque ningún warning.
- Sigue haciendo un commit por cada ejercicio.

 Este documento no incluye las soluciones a propósito, igual que en los bloques anteriores: la idea es que compruebes tú mismo si el resultado es el esperado.

3.1 · Listas

EJERCICIO 1 · Lista de piezas [Fácil]

Declara una lista `tipos_pieza: list` con los 6 nombres de tipo de pieza (`str`). Imprime cuántos elementos tiene y cuál es el tercero.

EJERCICIO 2 · Trocear la apertura [Fácil]

Declara una lista con 8 movimientos de ejemplo (`str`, como `"e2e4"`). Usando slicing, muestra los tres primeros y los tres últimos.

EJERCICIO 3 · Índices al límite [Medio]

Declara una lista de 8 elementos (las letras de columna). Imprime lo que devuelven los índices `-1`, `0` y `7`. Añade un comentario explicando por qué el índice `8` daría error.

3.2 · Métodos de listas

EJERCICIO 4 · Piezas capturadas [Fácil]

Parte de una lista vacía. Usa `append()` para añadir 4 nombres de piezas capturadas. Después, usa `pop()` para quitar la última y guarda el resultado en una variable. Imprime la lista final y la pieza que quitaste.

EJERCICIO 5 · Ordena los valores [Medio]

Declara una lista con los valores numéricos de las piezas (sin nombres, solo los números: `1`, `3`, `3`, `5`, `9`). Ordénala con `sort()` y muéstrala; después usa `reverse()` sobre la misma lista y muéstrala de nuevo.

EJERCICIO 6 · Arregla el append [Fácil — depuración]

Este fragmento pretende "pegar" dos listas, pero `append()` no hace eso. Corrígelo:

```
piezas_blancas: list = ["rey", "dama"]
piezas_negras: list = ["rey", "dama"]
piezas_blancas.append(piezas_negras)
print(piezas_blancas) # ¿de verdad quieres esto?
```

3.3 · Listas anidadas: matrices

EJERCICIO 7 · Matriz de ceros [Medio]

Con un `for`, crea una matriz 4×4 donde todas las casillas valgan 0. Imprímela completa usando dos bucles `for` anidados (una fila por línea).

EJERCICIO 8 · La diagonal de una matriz [Medio]

Declara una matriz 3×3 con los números del 1 al 9. Con dos bucles `for` anidados y un `if`, imprime solo los elementos de la diagonal principal (donde `fila == columna`).

EJERCICIO 9 · Suma de fila y columna [Reto]

Declara una matriz 4×4 de números enteros a tu elección. Calcula, con un `for` y un acumulador, la suma de todos los valores de la fila 0; después, con otro `for`, la suma de todos los valores de la columna 0. Imprime ambas sumas.

3.4 · Tuplas

EJERCICIO 10 · Coordenadas de piezas [Fácil]

Declara tres tuplas (`fila`, `columna`) para tres piezas distintas de tu elección. Imprime cada una junto con el nombre de la pieza correspondiente.

EJERCICIO 11 · ¿Por qué falla? [Fácil — depuración]

Este fragmento falla. Explica en un comentario por qué, y corrígelo (si necesitas modificar el valor, plantéate si en realidad querías una lista en vez de una tupla):

```
posicion_rey: tuple = (0, 4)
posicion_rey[1] = 5 # el rey se mueve a la columna 5
```

3.5-3.6 · Diccionarios

EJERCICIO 12 · Diccionario de valores [Fácil]

Crea un diccionario con 3 claves (nombres de piezas) y sus valores numéricos como valor. Imprime el valor de una de ellas accediendo directamente por su clave.

EJERCICIO 13 · Actualiza el diccionario [Medio]

Sobre el diccionario del ejercicio anterior: añade una clave nueva, modifica el valor de una que ya existía, y elimina una tercera con `del`. Imprime el diccionario después de cada paso.

EJERCICIO 14 · Usa .get() con teclado [Medio]

Declara un diccionario `pieza_en: dict` con 3 casillas ocupadas. Pide una casilla por teclado y muestra su contenido con `.get(casilla, "Sin pieza")`, sin usar `if` ni `try/except`.

EJERCICIO 15 · Recorre con .items() [Medio]

Declara un diccionario con 4 casillas y sus piezas. Con un `for` y `.items()`, imprime cada casilla junto a su pieza, una por línea.

3.7 · Lista de listas o diccionario

EJERCICIO 16 · Las dos versiones [Reto]

Implementa un mini-tablero 3×3 con una única pieza en el centro (posición (1,1) o clave "b2", según la versión): primero usando una lista de listas, después usando un diccionario. Imprime el resultado de las dos formas y comprueba que representan lo mismo.

3.8 · Errores comunes

EJERCICIO 17 · Arregla el IndexError [Fácil — depuración]

Corrige este fragmento para que no falle:

```
columnas: list = ["a", "b", "c", "d", "e", "f", "g", "h"]
ultima_columna: str = columnas[8]
print(ultima_columna)
```

EJERCICIO 18 · Arregla el KeyError [Fácil — depuración]

Corrige este fragmento para que no falle, aunque "h8" no sea una clave del diccionario:

```
pieza_en: dict = {"e1": "rey_blanco"}
print(pieza_en["h8"])
```

Reto integrador

EJERCICIO 19 · Medio tablero [Reto]

Crea una matriz 8×8 de casillas vacías ("·"). Usando slicing sobre la lista externa (sin bucles adicionales para seleccionar las filas), imprime solo la mitad inferior del tablero (filas correspondientes a los índices 0 a 3).

EJERCICIO 20 · Primer boceto de la Entrega 1 [Reto]

En la estructura de datos que hayas elegido para tu Entrega 1 (lista de listas o diccionario), coloca las 8 piezas de la primera fila blanca en sus casillas correctas: torre, caballo, alfil, dama, rey, alfil, caballo, torre — usando los símbolos Unicode de la tabla del Bloque 2. Imprime el resultado.

💡 El ejercicio 20 es, literalmente, el primer paso de tu Entrega 1: si te sale bien aquí, ya tienes una cuarta parte del tablero completo hecho.